

Frontend and UI Technologies Documentation

Basics to Practical Understanding with Examples

Core idea: Frontend technologies help us build the visible part of an application, handle user interaction, connect with backend APIs, and quickly create user interfaces or dashboards for real-world projects.

Prepared for	Frontend and UI Technologies Learning
Focus Areas	HTML, CSS, JavaScript, Node.js, Streamlit
Purpose	Academic submission and interview-ready revision
Prepared by	Parth Goyal

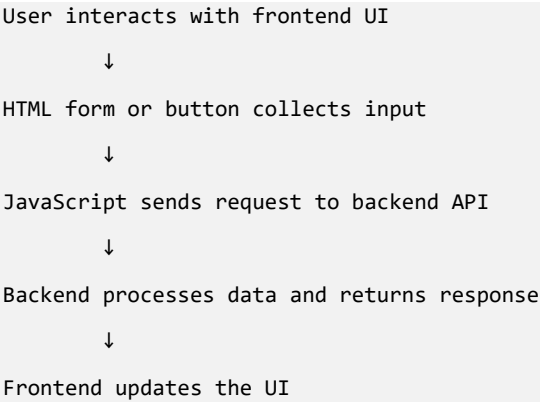
This document is written in a practical, topic-wise format so that it can be used for revision, project explanation, and viva/interview preparation.

1. Overview of Frontend and UI Technologies

Frontend and UI technologies are used to build the part of an application that users can see and interact with. In modern development, a frontend interface is not limited to static pages. It also collects user input, validates forms, sends API requests, receives backend responses, and displays dynamic data.

Technology	Main Role	Example Use
HTML	Creates webpage structure	Headings, forms, buttons, tables
CSS	Styles the page and controls layout	Colors, spacing, responsiveness, cards
JavaScript	Adds logic and interactivity	DOM changes, events, API calls
Node.js	Runs JavaScript outside the browser	Backend APIs, tools, packages
Streamlit	Creates Python-based UI quickly	ML demos, dashboards, CSV analysis

Basic frontend-backend flow



2. HTML Basics

HTML stands for HyperText Markup Language. It is used to create the structure of a webpage. It defines what should appear on the page, such as headings, paragraphs, links, images, forms, tables, and sections.

2.1 Basic HTML Structure

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Page</title>
</head>
<body>
  <h1>Hello World</h1>
  <p>This is my first webpage.</p>
</body>
</html>
```

Tag	Purpose
-----	---------

Frontend and UI Technologies Documentation

<!DOCTYPE html>	Tells browser that this is an HTML5 document
<html>	Root element of the HTML page
<head>	Contains metadata, title, CSS links and scripts
<body>	Contains visible content shown to the user
<title>	Sets browser tab title

2.2 Common HTML Elements

```
<h1>Main Heading</h1>
<p>This is a paragraph.</p>
<a href="https://example.com">Open Link</a>

<button>Click Me</button>
<input type="text" placeholder="Enter your name">
```

2.3 Forms in HTML

Forms are important because they collect user input. In real applications, JavaScript collects this data and sends it to backend APIs.

```
<form>
  <input type="email" placeholder="Enter email" required>
  <input type="password" placeholder="Enter password" required>
  <button type="submit">Login</button>
</form>
```

2.4 Semantic HTML

Semantic HTML means using meaningful tags that clearly describe the purpose of the content. It improves readability, accessibility and SEO.

```
<header>Website Header</header>
<nav>Navigation Menu</nav>
<main>Main Content</main>
<section>One Section</section>
<footer>Footer Content</footer>
```

3. CSS: Styling and Layout

CSS stands for Cascading Style Sheets. It is used to style HTML elements. CSS controls colors, fonts, spacing, layout, responsiveness and overall presentation of a webpage.

3.1 Ways to Add CSS

Method	Example	Usage
Inline CSS	Hello	Quick but not recommended for large projects

Internal CSS	<code><style> h1 { color: blue; } </style></code>	Useful for small single-page demos
External CSS	<code><link rel="stylesheet" href="style.css"></code>	Best practice for clean and reusable code

3.2 Selectors

```
/* Element selector */
h1 {
  color: blue;
}

/* Class selector */
.card {
  padding: 20px;
}

/* ID selector */
#main-title {
  color: purple;
}
```

Selector Type	Syntax	Meaning
Element	<code>h1</code>	Targets all h1 elements
Class	<code>.card</code>	Targets elements with class="card"
ID	<code>#main-title</code>	Targets element with id="main-title"
Universal	<code>*</code>	Targets all elements

3.3 Box Model

The CSS box model explains how every element is treated as a box. It includes content, padding, border and margin.

Content → Padding → Border → Margin

```
.card {
  width: 300px;
  padding: 20px;
  border: 1px solid #ccc;
  margin: 30px;
  box-sizing: border-box;
}
```

Property	Meaning
Content	Actual text or image inside the element

Padding	Space between content and border
Border	Boundary around the element
Margin	Space outside the element

3.4 Flexbox and Grid

Flexbox is used mainly for one-dimensional layouts, either row or column. Grid is used for two-dimensional layouts with rows and columns.

```
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 20px;
}

.cards {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
```

Flexbox	Grid
One-dimensional layout	Two-dimensional layout
Best for alignment and navbars	Best for dashboards and galleries
Works in row or column	Works with rows and columns together

3.5 Responsive Design

Responsive design means the webpage should adjust properly on mobile, tablet and desktop screens. Media queries, flexible widths, Flexbox and Grid help create responsive layouts.

```
@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
}
```

4. JavaScript: Logic and Interactivity

JavaScript is used to add logic, interactivity and dynamic behavior to webpages. It can handle button clicks, validate forms, change HTML content, call backend APIs and update the UI.

4.1 Variables and Data Types

```
let name = "Parth"; // changeable variable
```

Frontend and UI Technologies Documentation

```
const college = "GLA University"; // fixed value

let age = 21;

let isLoggedIn = true;

let user = {
  name: "Parth",
  email: "parth@gmail.com"
};

let skills = ["HTML", "CSS", "JavaScript"];
```

4.2 let, const and var

Keyword	Use
let	Block-scoped variable whose value can be changed
const	Block-scoped variable whose value should not be reassigned
var	Older function-scoped variable, generally avoided in modern JS

4.3 Functions and Arrow Functions

```
function add(a, b) {
  return a + b;
}

const multiply = (a, b) => {
  return a * b;
};

console.log(add(10, 20));
console.log(multiply(5, 4));
```

4.4 DOM Manipulation

DOM stands for Document Object Model. It represents the HTML document as JavaScript objects. JavaScript can access and modify HTML content, styles and classes using DOM methods.

```
<h1 id="title">Old Title</h1>
<button id="btn">Change Title</button>

<script>
const title = document.getElementById("title");
const button = document.getElementById("btn");

button.addEventListener("click", function () {
```

```
title.innerText = "New Title";  
title.style.color = "blue";  
});  
</script>
```

4.5 Form Handling and API Call

```
const form = document.getElementById("loginForm");  
  
form.addEventListener("submit", async function (event) {  
  event.preventDefault();  
  
  const email = document.getElementById("email").value;  
  const password = document.getElementById("password").value;  
  
  const response = await fetch("http://localhost:8000/login", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json"  
    },  
    body: JSON.stringify({ email, password })  
  });  
  
  const data = await response.json();  
  console.log(data);  
});
```

Important terms: `event.preventDefault()` stops the default form reload, `JSON.stringify()` converts a JavaScript object into JSON string, and `response.json()` converts backend JSON response into a JavaScript object.

5. Node.js: JavaScript Outside the Browser

Node.js is a JavaScript runtime environment that allows JavaScript to run outside the browser. It is commonly used for backend servers, REST APIs, command-line tools, real-time applications and development tooling.

5.1 Why Node.js is Useful

Feature	Explanation
Runs JS outside browser	Allows JavaScript on server or terminal
Uses V8 engine	Executes JavaScript code efficiently
npm ecosystem	Provides thousands of reusable packages
Non-blocking I/O	Handles many requests efficiently
Express.js support	Makes REST API development easier

5.2 Node.js vs Browser JavaScript

Browser JavaScript	Node.js
Runs inside browser	Runs outside browser on system/server
Can access DOM using document/window	Can access files, OS modules and servers
Used for UI interaction	Used for backend APIs and tools
Example: button click	Example: Express API server

5.3 npm and package.json

npm stands for Node Package Manager. It is used to install and manage packages in Node.js projects. package.json stores project metadata, scripts and dependencies.

```
npm init -y
npm install express

{
  "name": "node-demo",
  "version": "1.0.0",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

5.4 Basic Node.js Server

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello from Node.js server");
});

server.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

5.5 Express.js API Example

Express.js is a web framework for Node.js. It makes routing, middleware, request handling and JSON responses easier.

```
const express = require("express");

const app = express();
```


Frontend and UI Technologies Documentation

```
app.use(express.json());

let users = [
  { id: 1, name: "Parth", email: "parth@gmail.com" },
  { id: 2, name: "Aman", email: "aman@gmail.com" }
];

app.get("/users", (req, res) => {
  res.json(users);
});

app.post("/users", (req, res) => {
  const newUser = {
    id: users.length + 1,
    name: req.body.name,
    email: req.body.email
  };

  users.push(newUser);
  res.status(201).json(newUser);
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

5.6 Express Middleware, Path and Query Parameters

```
// Middleware example
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});

// Path parameter: /users/10
app.get("/users/:id", (req, res) => {
  const userId = Number(req.params.id);
  res.json({ userId });
});

// Query parameter: /products?category=mobile
```

```
app.get("/products", (req, res) => {  
    const category = req.query.category;  
    res.json({ category });  
});
```

6. Streamlit: Rapid UI Development with Python

Streamlit is a Python framework used to quickly build interactive web apps, dashboards and machine learning demos without writing complex frontend code. It is especially useful for AI/ML projects, data analysis, CSV dashboards and project demonstrations.

6.1 Why Streamlit is Useful

Use Case	Explanation
ML demo app	Take input, run model prediction and show result
Dashboard	Display tables, charts, metrics and filters
CSV analyzer	Upload dataset and perform EDA
AI prototype	Build chatbot or LLM demo quickly
Academic project demo	Present Python project with interactive UI

6.2 First Streamlit App

```
import streamlit as st  
  
st.title("My First Streamlit App")  
st.write("Hello, this app is built using Python and Streamlit.")  
  
# Run command  
streamlit run app.py
```

6.3 Common Streamlit Widgets

```
import streamlit as st  
  
name = st.text_input("Enter your name")  
age = st.number_input("Enter your age", min_value=1, max_value=100)  
skill = st.selectbox("Choose a skill", ["Python", "FastAPI", "JavaScript"])  
score = st.slider("Select score", 0, 100, 50)  
  
if st.button("Submit"):  
    st.success("Form submitted successfully")  
    st.write({"name": name, "age": age, "skill": skill, "score": score})
```

6.4 Streamlit Forms

Forms are useful because input processing happens only when the submit button is clicked. This avoids unnecessary processing on every widget change.

Frontend and UI Technologies Documentation

```
import streamlit as st

with st.form("registration_form"):
    name = st.text_input("Name")
    email = st.text_input("Email")
    course = st.selectbox("Course", ["B.Tech", "M.Tech", "MBA"])
    submitted = st.form_submit_button("Register")

if submitted:
    if name and email:
        st.success("Registration successful")
    else:
        st.error("Please fill all required fields")
```

6.5 CSV Upload and Dashboard Example

```
import streamlit as st
import pandas as pd

st.title("CSV Analyzer")

uploaded_file = st.file_uploader("Upload CSV file", type=["csv"])

if uploaded_file is not None:
    df = pd.read_csv(uploaded_file)
    st.subheader("Dataset Preview")
    st.dataframe(df.head())

    st.subheader("Dataset Shape")
    st.write(df.shape)

    st.subheader("Missing Values")
    st.write(df.isnull().sum())

    st.subheader("Summary Statistics")
    st.write(df.describe())
```

6.6 Streamlit ML Prediction Flow

```
Train ML model in Python
↓
Save model using pickle or joblib
↓
```

Frontend and UI Technologies Documentation

Load model in Streamlit app

↓

Take user input through widgets

↓

Call `model.predict()`

↓

Display prediction result

```
import streamlit as st
```

```
import pickle
```

```
import numpy as np
```

```
model = pickle.load(open("model.pkl", "rb"))
```

```
st.title("Simple ML Prediction App")
```

```
age = st.number_input("Age", min_value=1, max_value=100)
```

```
salary = st.number_input("Salary", min_value=0)
```

```
if st.button("Predict"):
```

```
    input_data = np.array([[age, salary]])
```

```
    prediction = model.predict(input_data)
```

```
    st.success(f"Prediction: {prediction[0]}")
```

6.7 Session State

Streamlit reruns the script from top to bottom whenever a user interacts with a widget. Session state is used to preserve values across reruns, such as login state, chat history or counters.

```
import streamlit as st
```

```
if "count" not in st.session_state:
```

```
    st.session_state.count = 0
```

```
if st.button("Increase"):
```

```
    st.session_state.count += 1
```

```
st.write("Count:", st.session_state.count)
```

7. Important Comparisons

7.1 Node.js vs Streamlit

Node.js	Streamlit
JavaScript runtime outside browser	Python framework for quick UI apps
Used for backend APIs and tools	Used for dashboards and ML demos
Uses npm and Express.js	Uses Python libraries like pandas, sklearn
Better for server-side applications	Better for rapid prototyping
Returns APIs/JSON responses	Creates direct web interface

7.2 FastAPI vs Streamlit

FastAPI	Streamlit
Backend API framework	Frontend-like Python UI framework
Returns JSON responses	Displays UI directly to users
Used for APIs, auth and database logic	Used for demos, dashboards and ML interface
Good for production backend	Good for quick project demo

7.3 Streamlit with FastAPI Concept

```
Streamlit UI
↓ sends request
FastAPI backend
↓ processes prediction/database logic
JSON response
↓
Streamlit displays result

import streamlit as st
import requests

value = st.number_input("Enter value")

if st.button("Predict"):
    response = requests.post(
        "http://localhost:8000/predict",
        json={"value": value}
    )
```

```
result = response.json()

st.write(result)
```

8. Final Summary

Frontend and UI technologies are important because they help convert backend logic and data into a usable interface. HTML creates structure, CSS adds design, and JavaScript adds interactivity and API integration. Node.js expands JavaScript beyond the browser and allows it to be used for backend development and tooling. Streamlit allows Python developers to build dashboards, ML demos and interactive apps quickly.

A practical developer should understand how these technologies work together: HTML/CSS build the interface, JavaScript handles user actions and API calls, Node.js can create JavaScript-based backend services, and Streamlit can quickly present Python or ML projects through a simple UI.